

AE646 - Interim Report (Stage 2)

Operator Learning for Parametric Darcy Flow: FNO vs MLP Baseline

Team - PINNacles

18th September 2026

1 Introduction

Traditional numerical solvers such as Finite Difference Methods (FDM) and Finite Element Methods (FEM) require re-solving the full PDE for every new set of parameters, imposing prohibitive computational cost when thousands of queries are needed (e.g., uncertainty quantification, inverse design, real-time control). Neural operators learn a mapping between infinite-dimensional function spaces and amortise this cost over many queries.

This project evaluates two architectures for the 2D parametric Darcy flow problem:

- **MLP baseline** - a fully-connected network with no structural inductive bias.
- **Fourier Neural Operator (FNO)** [1] - a resolution-invariant architecture whose trainable weights reside in the frequency domain.

Both are trained to map the permeability field $\kappa(x, y)$ and spatial coordinates (x, y) to the pressure field $u(x, y)$ satisfying the 2D Darcy equation:

$$-\nabla \cdot (\kappa(x, y) \nabla u(x, y)) = f(x, y), \quad (x, y) \in [0, 1]^2, \quad (1)$$

with homogeneous Dirichlet boundary conditions. The forcing $f = 1$ is constant; the permeability $\kappa \in \{0.1, 1.0\}$ is piecewise-constant, randomly distributed on a 64×64 grid.

Scope of this report. At Stage 2 the data pipeline is complete, the MLP baseline is implemented and tuned through a series of ablations, and the FNO is implemented and trained with preliminary quantitative results. The remaining evaluation and analysis planned for Stage 3 is listed in Section 9.

2 Dataset Setup

2.1 Source

We use the PDEBench dataset [2] (DOI: 10.18419/darus-2986), a benchmark suite for scientific machine learning. The 2D Darcy flow split provides 10,000 samples at 128×128 resolution.

2.2 Preprocessing

Each sample consists of:

- **Input:** $\kappa(x, y)$ subsampled from 128×128 to 64×64 by taking every second grid point (literature-standard subsampling; PDEBench’s own loaders do the same).
- **Grid coordinates:** $(x, y) \in [0, 1]^2$ concatenated channel-wise to give a 3-channel input tensor of shape $64 \times 64 \times 3$.
- **Target:** pressure $u(x, y)$ at 64×64 .

From the 10,000 sample archive we draw a reproducible (seed 42), non-overlapping subset: a 1,000 sample train & val pool, split 900 train / 100 validation, and 200 further samples held out entirely throughout training and used only for final evaluation. These counts match those

used in the original FNO work (Li et al. 2021), so our numbers are directly comparable to the literature. Data are loaded in mini-batches of 16 and normalised with training-set statistics only.

2.3 Why Only a 1,000 Sample Training Pool?

The original FNO Darcy experiments use $\sim 1,000$ training samples; matching this keeps the comparison with published numbers clean and the training cost modest. 100 validation samples give a stable loss estimate for checkpoint selection, and the 200 sample test set is large enough for generalisation statistics with standard error $\lesssim 0.003$.

2.4 Exploratory Data Analysis

Before modelling we characterised the 1,200 real samples used across the three splits (Figure 1).

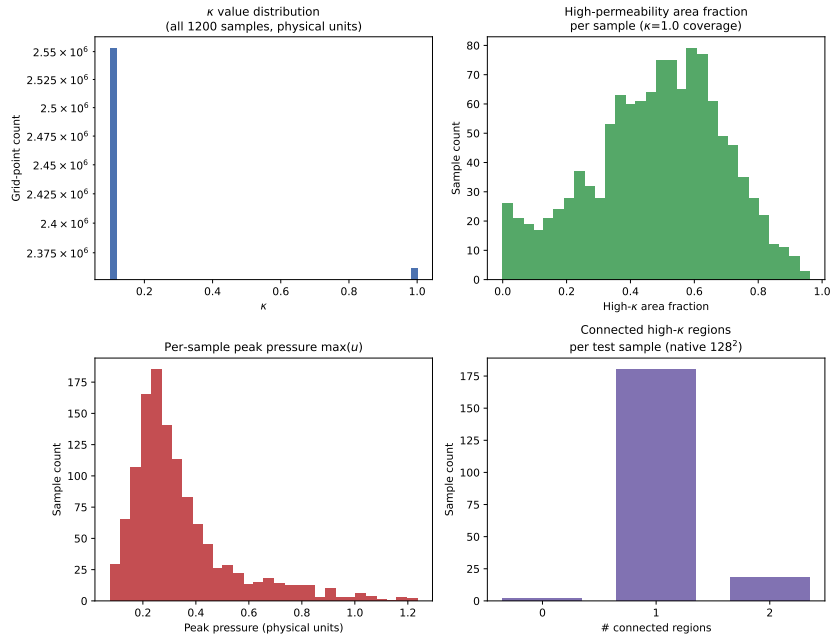


Figure 1: Dataset characterisation. κ is exactly bimodal at $\{0.1, 1.0\}$; the high-permeability area fraction varies widely (mean 0.48 ± 0.21); peak pressure has mean 0.34 ± 0.19 (physical units). At native 128×128 , 180/200 test samples contain exactly one connected high- κ region, 18/200 contain two, and 2/200 are exactly uniform ($\kappa \equiv 0.1$, zero regions).

Three observations shape the modelling. (i) The permeability alphabet is binary, so sample-to-sample diversity comes from the *shape and area* of the high-permeability region, not from its value. (ii) Samples are almost always a single blob (multi-region fields are rare), so the dataset is geometrically simple but the pressure amplitude varies strongly across samples (peak pressure ranges from about 0.1 to over 1.0). (iii) A few samples are exactly uniform in κ ; these are rare edge cases that we track separately in the results (Section 6).

2.5 Preprocessing Ablation (MLP)

To check that the preprocessing choices are sensible rather than assumed, we varied one choice at a time on the MLP baseline and retrained from scratch with three seeds each, on exactly the same split and target (so the test error is directly comparable). Table 1 and Figure 2 give the results.

Table 1: MLP preprocessing ablation: test mean relative L_2 (200 samples), mean $\pm$ std over 3 seeds.

Preprocessing variant	Test mean rel. L_2
Reference: standardised inputs and targets, + coordinates	0.0842 ± 0.0002
No coordinate channels (κ only)	0.0843 ± 0.0011
No input normalisation (raw κ)	0.0864 ± 0.0018
No target normalisation (raw pressure)	0.0917 ± 0.0012

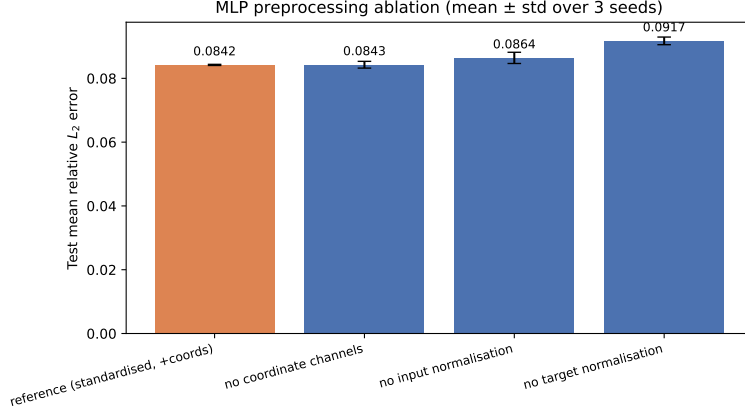


Figure 2: MLP preprocessing ablation (mean $\pm$ std over 3 seeds).

Normalising the target is the choice that matters (+9% error without it, several seed-standard-deviations outside the reference). Input normalisation helps slightly. Coordinate channels make no measurable difference for the MLP (its flattened input already fixes each pixel’s position); we keep them because they are the standard FNO input convention and give the two models identical inputs.

3 Model Architectures

3.1 MLP Baseline

The MLP baseline flattens the $64 \times 64 \times 3$ input into a vector of 12,288 features and applies three hidden layers of width 2048 with GELU activations:

$$\text{MLP} : \mathbb{R}^{12288} \xrightarrow{2048} \xrightarrow{2048} \xrightarrow{2048} \mathbb{R}^{4096} \rightarrow \mathbb{R}^{64 \times 64},$$

with a final linear projection. Total parameters: ≈ 42 M. The MLP is fixed to the training resolution (64×64) and cannot generalise to other grid sizes.

3.2 Fourier Neural Operator (FNO)

The FNO [1] processes the input as a $64 \times 64 \times 3$ field tensor through five stages:

1. **Lifting**: a pointwise linear layer maps 3 input channels to 64 hidden channels.
2. **Four Spectral Convolution Blocks**: each block computes (i) a 2D FFT of the 64-channel field, (ii) retains only the lowest 12 Fourier modes in each direction (12×12 complex-valued coefficients), (iii) multiplies by learned complex weights of shape $64 \times 64 \times 12 \times 12$, (iv) applies an inverse FFT, and (v) adds a pointwise local convolution bypass (equivalent to a 1×1 convolution).

3. **Projection:** two linear layers ($64 \rightarrow 128 \rightarrow 1$) map hidden channels to the scalar output.

Because the learned weights operate in frequency space (not on a fixed grid), the same trained network can in principle be evaluated at other grid resolutions, a property we plan to test in Stage 3. Total parameters: ≈ 4.7 M.

4 Training Protocol

Both models are trained with:

- **Loss:** mean-squared error between the predicted and true standardised pressure fields.
- **Reported metric:** relative L_2 error $\|\hat{u} - u\|_2 / \|u\|_2$ per sample, computed after converting back to physical units (Section 8).
- **Optimiser:** AdamW (decoupled weight decay 10^{-4}), peak learning rate 10^{-3} , with PyTorch’s `CosineAnnealingLR` ($T_{\max} = 100$) advanced once per mini-batch step. The learning rate therefore cycles between 10^{-3} and 0 every 200 optimiser steps (about 3.5 epochs, ~ 28 cycles per run), i.e. a warm-restart-like schedule rather than a single decay; it is identical for every model and ablation, so comparisons are like-for-like.
- **Batch size / epochs:** 16 / 100. No early stopping: every run completes its full epoch budget and the checkpoint with the lowest validation relative- L_2 error is retained (best epochs 92 for the FNO and 99 for the MLP).

5 MLP Baseline Ablation: Depth and Optimiser

The reported baseline (3 hidden layers of 2048, AdamW) was compared against shallower and differently-optimised variants, each retrained from scratch on the same data, split, loss and schedule (`src/ablation_mlp.py`). Table 2 and Figure 3 report whatever the test set gives.

Table 2: MLP baseline ablation, 200 test samples, single run per configuration (`results/ablation_mlp.json`).

Configuration	Params	Mean rel. L_2	Median
1 hidden layer (2048), AdamW	33.6 M	0.1008	0.0892
2 hidden layers (2048 $\times$ 2), AdamW	37.8 M	0.0796	0.0649
3 hidden layers (2048 $\times$ 3), AdamW (final)	42.0 M	0.0857	0.0700
3 hidden layers (2048 $\times$ 3), SGD+momentum	42.0 M	0.1209	0.1091

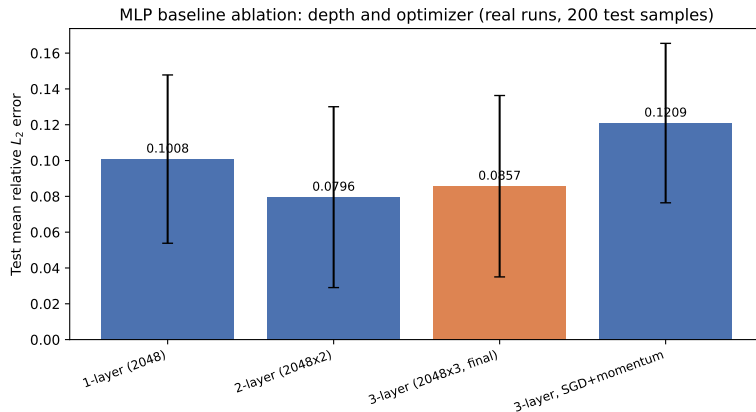


Figure 3: MLP baseline ablation (error bars: one standard deviation across the 200 test samples).

Two clear conclusions: a single hidden layer is too weak, and SGD with momentum is much worse than AdamW (0.1209 vs. 0.0857), which justifies the optimiser choice. Depth beyond two layers brings no benefit at this dataset size: the 2-layer variant is marginally better than the 3-layer one, but a gap of ~ 0.006 is of the same order as the run-to-run variation we measured (~ 0.004 , Section 8), so we treat 2 and 3 layers as equivalent. We keep the 3-layer network as the baseline because it was fixed before this study; it is therefore a fair, not a deliberately weak, comparison point for the FNO.

6 Preliminary Quantitative Results

All metrics are computed on the held-out 200-sample test set. Error is relative L_2 : $\epsilon_{\text{rel}} = \|\hat{u} - u\|_2 / \|u\|_2$ in physical units.

Table 3: Test-set performance at training resolution (64×64). Lower is better.

Model	Params	Mean	Median	Std	Min	Max
MLP baseline	42.0 M	0.0820	0.0695	0.0456	0.0325	0.3367
FNO	4.7 M	0.0521	0.0398	0.0451	0.0155	0.3467

Key observations:

1. The FNO achieves 36% lower mean error than the MLP while using $9\times$ fewer parameters, demonstrating the advantage of spectral inductive biases for smooth PDE solutions.
2. Both models have right-skewed error distributions: most samples have errors well below 0.1, while a small fraction ($\lesssim 5\%$) pushes the error toward 0.3 (Figure 4). The maximum error is similar for the two models, so the FNO’s advantage is in the bulk of the distribution rather than the tail.
3. The single worst FNO test sample is not a complex field but the opposite: a perfectly uniform permeability field (Figure 5, bottom row).

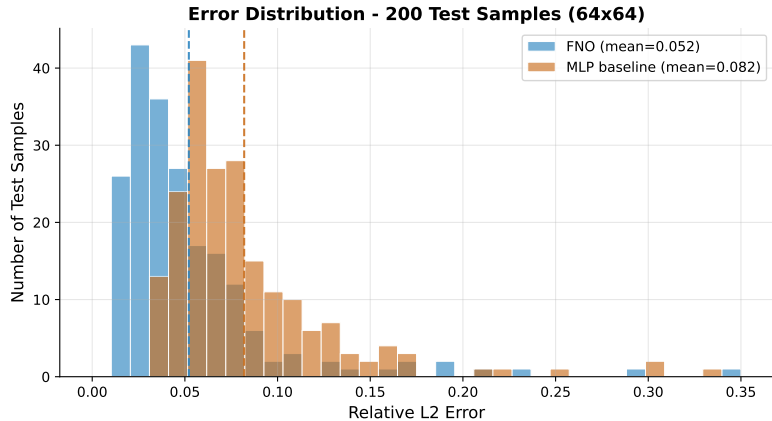


Figure 4: Error distribution across 200 test samples. Dashed vertical lines indicate per-model means. FNO errors are concentrated at lower values; the MLP’s distribution is shifted right.

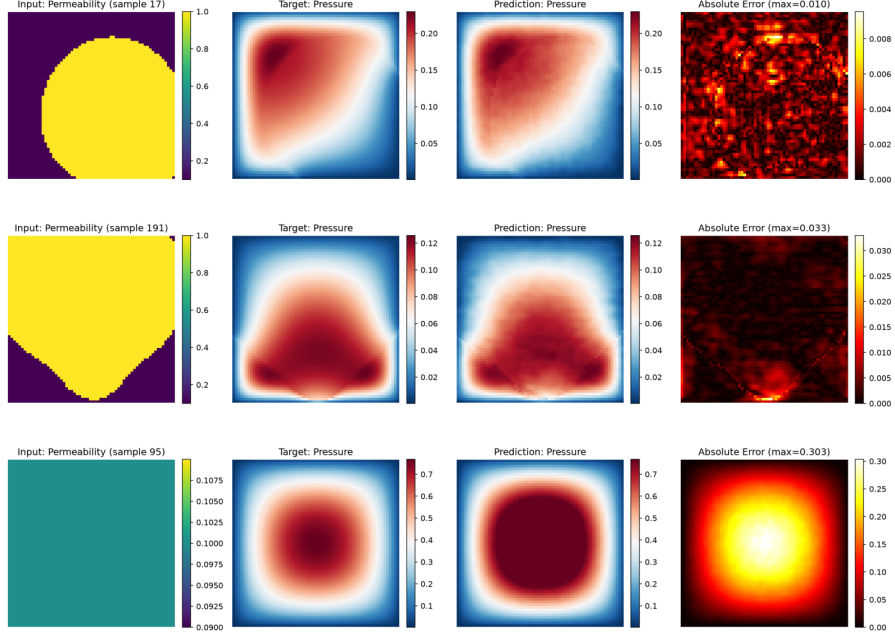


Figure 5: FNO predictions for the best (sample 17, error 0.016), a median-error (sample 191, error 0.040) and the worst (sample 95, error 0.347) test sample. Columns: input κ , target pressure, predicted pressure, absolute error. The worst case has exactly uniform $\kappa \equiv 0.1$: the true pressure is a smooth bump, but the FNO over-predicts its amplitude (the prediction saturates the colour scale of the target) — a failure mode specific to this degenerate input.

7 Code Quality and Reproducibility

The codebase is organised into four directories:

- `src/` - `download_data.py` and `preprocess.py` (checksum-verified download, split and normalisation), `models.py` (MLPBaseline, SpectralConv2d, FNO2d), `train.py` (training loop; keeps the best-validation checkpoint), `evaluate.py` (test metrics and sample plots), `eda.py`, `ablation_mlp.py`, `ablation_preprocess.py`.
- `configs/` - `fno.yaml`, `mlp.yaml`.
- `results/` - one `run_00x/` per model with its `eval_metrics.json`, plus the ablation and EDA outputs and .
- `tests/` - the `pytest` suite.

A `pytest` suite covers model shapes, the relative- L_2 metric, and data loading; all results are committed. Training and evaluation reproduce with:

```
python3 src/download_data.py
python3 src/preprocess.py
python3 src/train.py --config configs/fno.yaml
python3 src/evaluate.py --config configs/fno.yaml \
    --checkpoint results/run_001/best_model.pt
```

Platform and notebook. During development each step (data preparation, training, evaluation, dataset analysis, ablations) was a separate Python script. We later combined them into one Jupyter notebook, `PINNacles_Stage2_Notebook.ipynb`, included in the code archive, which runs the whole pipeline and shows the results inline.

8 Issues Encountered

1. **Metric convention.** Relative L_2 must be computed in physical units: on standardised fields the subtracted mean changes $\|u\|_2$, so the number is not comparable with the literature. All reported errors are de-normalised, and the metric is unit-tested.
2. **Uniform-permeability samples.** Exactly uniform κ fields (2 of the 200 test samples) are a degenerate input on which the FNO over-predicts the pressure amplitude; one of them is the worst test sample (error 0.347, Figure 5). We report this failure openly rather than excluding those samples.
3. **Compute constraints.** Loading the 1.3GB raw file and training the MLP and the FNO (plus the ablations) was moved to the lab GPU machine.
4. **Run-to-run variance.** The first training runs did not fix the model-initialisation seed; re-training the identical MLP gives errors differing by ~ 0.004 (0.0820 reported vs. 0.0857 in Section 5), which we treat as the noise floor: only larger gaps are interpreted. Training now seeds Python, NumPy and PyTorch, and the preprocessing ablation uses 3 fixed seeds per variant.

9 Future Work (Stage 3)

The following are planned for the final stage:

1. **FNO scaling study.** Train larger FNO configurations (more width, Fourier modes and blocks) and vary modes and width systematically to map the accuracy-parameter trade-off against the MLP baseline.
2. **Resolution generalisation.** Evaluate the trained FNO zero-shot at the native 128×128 resolution using the native-resolution test fields already set aside during preprocessing, and compare it with an FNO trained directly at 128×128 .
3. **Computational cost.** Benchmark inference time of both models against the finite-difference reference solver, and profile the cost of the individual FNO operations (FFT, spectral multiplication, local convolution).
4. **Physical interpretation.** Quantify how the per-sample error relates to the geometry of the permeability field (number of regions, interface length, area fraction), including the uniform- κ failure cases identified above.
5. **Learning-rate schedule.** Compare the current per-step cosine cycling with standard per-epoch cosine annealing, to check how much of the remaining error is optimisation rather than model capacity.
6. **Final deliverables.** Final report, final presentation, and viva.

References

- [1] Z. Li et al., “Fourier Neural Operator for Parametric Partial Differential Equations,” *ICLR*, 2021.
- [2] M. Takamoto et al., “PDEBench: An Extensive Benchmark for Scientific Machine Learning,” *NeurIPS*, 2022. DOI: 10.18419/darus-2986.